

Documentation technique - Blacksmith AI

Description des fichiers du projet et contributions apportées durant le stage

Orange Innovation - Stage 2026

■ NOUVEAU : fichier créé durant le stage ■ MODIFIÉ : fichier existant, corrigé ou étendu
Sans badge : fichier préexistant, non modifié

1. Vue d'ensemble du projet

Blacksmith AI est un framework de tests d'intrusion automatisés basé sur une architecture multi-agents, construit sur LangChain et LangGraph. Il orchestre six agents spécialisés (Recon, ScanEnum, VulnMap, Exploit, PostExploit, Pentester) exécutant des commandes dans un conteneur Docker mini-Kali isolé du réseau cible. Le projet est localisé sur la machine rt-controler du laboratoire Orange sous `blacksmith_attck/blacksmithAI/`.

Deux fichiers de configuration coexistent selon l'environnement : `config.json` pour le modèle local `qwen3-5-9b-awq-4bit` (GPU du laboratoire), et `config_llmproxy.json` pour Claude Opus 4.6 via le proxy LLM d'Orange. Toute la configuration applicative est centralisée dans ces deux fichiers.

```
blacksmith_attck/blacksmithAI/
├─ main.py                      # Orchestrateur principal
├─ pentest.py                   # Agent standalone (CTF) [NOUVEAU]
├─ pentest_avant.py            # Version avant modifications
├─ pentest_avec_juste_searxng.py # Version test web_search uniquement
├─ logging_config.py           # Configuration des logs [NOUVEAU]
├─ build_hacktricks_rag.py      # Construction base RAG [MODIFIÉ]
├─ config.json                 # Config modèle local (qwen3)
├─ config_llmproxy.json        # Config proxy LLM Orange (Claude Opus 4.6)
├─ docker-compose.yml          # Services Docker [MODIFIÉ]
├─ Dockerfile                  # Image mini-Kali
├─ tools.md                    # Documentation des outils
├─ Implementations.md          # Notes d'implémentation
├─ hacktricks/                 # Contenu HackTricks (source RAG)
├─ store/                      # Base vectorielle ChromaDB
├─ searxng/                    # Configuration SearXNG
├─ mcp/                        # Répertoire MCP
├─ reports/                    # Rapports ATT&CK générés
├─ reports_10.0.0.3/           # Rapports session LiteLLM
├─ reports_10.0.0.10_.../      # Rapports session Vulnerability Map
├─ logs/
│   └─ blacksmith.log          # Logs applicatifs rotatifs
│   └─ exec_events.jsonl       # Journal JSONL des commandes [NOUVEAU]
├─ agents/
│   └─ base.py                 # Classe de base partagée [MODIFIÉ]
│   └─ base_avant.py           # Version avant modifications
│   └─ recon.py                # Agent reconnaissance [MODIFIÉ]
```

```

|   ├── scan_enum.py           # Agent scan & énumération [MODIFIÉ]
|   ├── vuln_map.py           # Agent vulnerability mapping [MODIFIÉ]
|   ├── exploit.py            # Agent exploitation [MODIFIÉ]
|   ├── post_exploit.py       # Agent post-exploitation [MODIFIÉ]
|   ├── pentester.py          # Agent orchestrateur [MODIFIÉ]
|   └── tools_doc/            # Documentation outils par agent
├── middleware/
|   └── attck_autotag.py       # Tagging ATT&CK automatique [NOUVEAU]
├── backends/
|   └── container_backend.py   # Persistance fichiers conteneur [NOUVEAU]
├── tools/
|   ├── tools.py              # Outils agents [MODIFIÉ]
|   ├── shell/                # Exécuteur shell du conteneur
|   ├── tools_avant.py        # Version initiale
|   ├── tools_avant_websearch.py # Après ajout web_search
|   ├── tools_avant_mitre.py  # Avant intégration MITRE finale
|   ├── tools.py              # Outils agents [MODIFIÉ]
|   └── mail_tool.py          # Outil email CTF [NOUVEAU]
└── utils/
    ├── vectors.py            # Interface ChromaDB [MODIFIÉ]
    ├── attck_report.py       # Rapport ATT&CK + déduplication [NOUVEAU]
    ├── cve_validator.py      # Valideur de CVE [NOUVEAU]
    ├── context_manager.py    # Gestionnaire de contexte [NOUVEAU]
    ├── exec_event_log.py     # Journal JSONL commandes [NOUVEAU]
    ├── exec_event_log_avant.py # Version avant modifications
    ├── ebpf_correlator.py    # Squelette corrélateur eBPF [NOUVEAU]
    └── loader.py             # Chargement configuration [MODIFIÉ]

```

2. Fichiers de configuration

config.json [MODIFIÉ]

Configuration principale pour le modèle local qwen3-5-9b-awq-4bit hébergé sur le GPU du laboratoire (endpoint <http://10.0.0.3/v1>).

- Ajout de la section searxng (url: <http://10.0.4.1:8888>, timeout, max_results)
- Ajout de la catégorie passwords_crypto dans les outils du conteneur (hashcat, john, crunch, cewl)
- Extension des catégories existantes : scanning, reconnaissance, post_exploit
- default_embedding_model fixé à qwen3-embedding-0-6b, paramètre lu par build_hacktricks_rag.py ET par tools.py pour garantir la cohérence des dimensions de vecteurs entre création et requête de la base RAG

config_llmproxy.json [NOUVEAU]

Configuration dédiée au proxy LLM d'Orange pour les tests avec Claude Opus 4.6 (endpoint <https://llmproxy.ai.orange/v1>). Permet de basculer d'environnement sans modifier config.json.

- Modèle : vertex_ai/claude-opus-4-6 via le proxy Orange

- Embedding : openai/text-embedding-3-large (dimension 1536), nécessite une reconstruction de la base RAG si on bascule depuis config.json (dimension 1024 avec qwen3-embedding-0-6b)
- context_size: 128 000 tokens
- Utilisé pour le test comparatif Claude vs Qwen documenté en section 5.4

3. Fichiers modifiés

`main.py` [MODIFIÉ]

Orchestrateur principal : initialise les agents, partage le report_builder entre eux, gère la session complète et affiche le rapport final.

- Initialisation partagée de ATTCKReportBuilder passé à chaque agent via report_builder= (consolidation des preuves inter-agents)
- Remplacement du TodoListMiddleware (erreurs de validation schema Pydantic avec le modèle quantifié) par SummarizationMiddleware à 70 %
- Ajout du CVEValidator après génération du rapport final
- configure_logging() appelé en tout premier, avant tout import applicatif
- recursion_limit porté à 1 000 ; timeout wall-clock désactivé par défaut

`agents/base.py` [MODIFIÉ]

Classe de base partagée par les six agents : initialise l'embedding model, la connexion LLM, et fournit les méthodes communes. Point central lu par tous les agents, c'est ici que le modèle d'embedding est instancié depuis config.json.

- Ajout du paramètre report_builder= dans create_deep_agent(), propagé à ATTCKAutoTagMiddleware et ContainerBackend
- Ajout de backend=ContainerBackend() pour la persistance des fichiers dans le conteneur pentest
- Ajout de ATTCKAutoTagMiddleware avec phase et agent_name par agent
- init_embedding_model() lit default_embedding_model depuis config.json, point de configuration unique garantissant la cohérence RAG

`agents/recon.py`, `scan_enum.py`, `vuln_map.py`, `exploit.py`, `post_exploit.py`, `pentester.py` [MODIFIÉ]

Six agents spécialisés, chacun responsable d'une phase du pentest. Les modifications sont identiques sur chacun et passent par base.py.

- Passage de report_builder=, phase= et agent_name= spécifiques à chaque agent
- Les agents héritent désormais des corrections de base.py (ContainerBackend, ATTCKAutoTagMiddleware) sans modification individuelle du code

`utils/vectors.py` [MODIFIÉ]

Interface ChromaDB pour la recherche de similarité dans la base RAG.

- Correction du bug d'indentation (mélange tabs/espaces) causant un comportement silencieusement incorrect
- Remplacement de `similarity_search()` par `similarity_search_with_relevance_scores()` avec seuil à 0,35
- Ajout du fil d'Ariane de section (h1 › h2 › h3) dans les résultats

`utils/loader.py` [MODIFIÉ]

Chargement et validation de la configuration depuis `config.json` ou `config_llmproxy.json` selon la variable d'environnement active.

- Prise en charge des deux fichiers de configuration (local vs proxy LLM)
- Validation des champs obligatoires au démarrage pour détecter les configurations incomplètes avant le lancement d'une session

`tools/`

Outils mis à disposition des agents. Trois versions intermédiaires documentent l'évolution des contributions : `tools_avant.py` (version initiale), `tools_avant_websearch.py` (après ajout de `web_search`), `tools_avant_mitre.py` (après ajout de `attck_tagger`, avant version finale).

- `pentest_shell`: timeout client tuple (10s connect, timeout+15s read) pour éviter les blocages indéfinis
- `pentest_shell` : troncature head+tail (8 000 + 8 000 chars) sur les sorties dépassant 20 000 caractères
- `_serialize_tool_result()` appliqué à tous les outils (résout l'AttributeError sur les retours dict bruts dans deepagents)
- Ajout de `web_search()` : appel SearXNG sur `http://10.0.4.1:8888`
- Ajout de `attck_tagger()` : outil de tagging manuel en complément du middleware automatique

`build_hacktricks_rag.py` [MODIFIÉ]

Script de construction de la base vectorielle RAG depuis le dossier `hacktricks/`. Le contenu HackTricks est présent localement dans le répertoire `hacktricks/` du projet ; la base vectorielle est stockée dans `store/`.

- Remplacement de la stratégie mono-passe par deux passes : `MarkdownHeaderTextSplitter` (`strip_headers=False`) puis `RecursiveCharacterTextSplitter` (`chunk_size=1000`, `overlap=150`)
- Filtre anti-chunks vides : chunks < 100 caractères éliminés
- Modèle d'embedding lu depuis `config.json` via `loader.py` (cohérence garantie entre création et requête)

`docker-compose.yml` [MODIFIÉ]

Configuration Docker du conteneur mini-Kali et des services associés. Le répertoire `searxng/` contient la configuration de l'instance SearXNG.

- Port mapping SearXNG: `localhost:8888` → `conteneur:8080`

4. Fichiers créés

`middleware/attck_autotag.py` [NOUVEAU]

Middleware structurel interceptant chaque appel `pentest_shell` pour appliquer automatiquement le tagging MITRE ATT&CK, sans dépendre de la compliance du modèle.

- Résolution via base de connaissances locale (~60 outils) avec fallback LLM
- Versions sync (`wrap_tool_call`) et async (`awrap_tool_call`) toutes deux implémentées — l'absence de l'async causait `NotImplementedError`
- `evidence_status='declared'` sur tous les événements

`backends/container_backend.py` [NOUVEAU]

Persistance des fichiers créés par les agents dans le conteneur Docker. Résout la perte des fichiers causée par le StateBackend mémoire par défaut de deepagents.

- Hérite de `AbstractBackend` (6 méthodes : `write`, `read`, `exists`, `delete`, `list`, `mkdir`)
- Encodage base64 pour les fichiers binaires
- Toutes les opérations s'exécutent via `pentest_shell` dans le contexte du conteneur

`utils/attck_report.py` [NOUVEAU]

Constructeur du rapport MITRE ATT&CK : centralise les preuves de tous les agents, gère la déduplication et génère les rapports dans `reports/`.

- Buffer de preuves partagé entre agents (instance unique passée via `report_builder=`)
- Déduplication par fenêtre glissante 5 min, max 2 appels identiques par fenêtre
- `get_recent_call_count()` (lecture seule) séparé de `record_command_execution()` (incrémementation) pour éviter les doubles comptages
- `get_evidence_text()` utilisé par `CVEValidator` pour la vérification croisée
- Rapports sauvegardés dans `reports/` avec horodatage

`utils/cve_validator.py` [NOUVEAU]

Valide les identifiants CVE du rapport final contre les preuves réelles du buffer d'évidence. Affiche un bandeau d'avertissement pour chaque CVE non vérifiée.

- Regex `CVE-\d{4}-\d{4,7}` appliquée sur le texte de synthèse
- Vérification contre `report_builder.get_evidence_text()` (preuves des agents, pas de l'orchestrateur)

- Validé en conditions réelles : 1 CVE signalée sur 2 citées lors d'un test sur LiteLLM 1.83.14

`utils/context_manager.py` [NOUVEAU]

Gestionnaire de contexte pour la fenêtre de tokens des agents : surveille le taux de remplissage et déclenche la summarisation avant d'atteindre la limite.

- Seuil de déclenchement configurable (défaut : 70 % de la fenêtre)
- Utilisé par SummarizationMiddleware dans main.py
- Résout les ContextWindowExceededError observées sur les sessions longues

`utils/exec_event_log.py` [NOUVEAU]

Journal JSONL horodaté de chaque commande exécutée via pentest_shell. Stocké dans logs/exec_events.jsonl. Constitue la clé de jointure pour la future corrélation eBPF.

- Entrée par commande : timestamp ISO 8601, commande, durée, exec_event_id (UUID)
- exec_event_id également stocké dans le buffer ATT&CK pour la jointure future
- Chemin configurable via EXEC_EVENT_LOG_PATH (défaut : logs/exec_events.jsonl)

`utils/ebpf_correlator.py` [NOUVEAU]

Squelette du corrélateur eBPF/Tetragon. Pose l'infrastructure pour faire passer les événements ATT&CK de declared à kernel_confirmed.

- Résolution container_name → container_id via l'API Docker locale
- Corrélation par fenêtre temporelle autour de chaque exec_event_id
- Non déployé — dépend d'une TracingPolicy Tetragon à configurer

`logging_config.py` [NOUVEAU]

Configuration centralisée et idempotente des logs. Doit être appelée en tout premier dans main.py et pentest.py.

- RotatingFileHandler : logs/blacksmith.log, 10 MB × 5 fichiers, niveau DEBUG
- StreamHandler console : WARNING uniquement
- Filtrage des bibliothèques tierces (chromadb, httpcore, urllib3, langsmith, httpx) au niveau WARNING
- Protection idempotente (_configured) contre les doubles appels

`mail_tool.py` [NOUVEAU]

Outil d'envoi de codes secrets par email, développé pour le CTF Quête de Plankton. Localisé à la racine du projet ou dans un répertoire dédié (non dans tools/). Disponible uniquement pour l'orchestrateur et pentest.py.

- Validation du format du code secret (hex) avant envoi
- Déduplication via journal JSONL local pour éviter les envois multiples

- Domaine de validation hardcodé : quete-de-plankton.ebfe.fr

5. Répertoires générés automatiquement

Ces répertoires et fichiers sont générés à l'exécution et ne font pas partie du code source du projet.

logs/blacksmith.log Logs applicatifs rotatifs (10 MB × 5). Contient DEBUG complet de toutes les sessions.

logs/exec_events.jsonl Journal JSONL des commandes shell exécutées. Une ligne par appel pentest_shell, avec exec_event_id.

store/ Base vectorielle ChromaDB contenant les embeddings HackTricks. À reconstruire si le modèle d'embedding dans config.json change.

reports/ Rapports ATT&CK générés en Markdown après chaque session.

reports_10.0.0.3/ Rapports des sessions de test sur le serveur LiteLLM.

reports_10.0.0.10.../ Rapports des sessions de Vulnerability Mapping.

hacktricks/ Contenu du dépôt HackTricks utilisé comme source RAG. À mettre à jour périodiquement pour les nouvelles techniques.

searxng/ Configuration de l'instance SearXNG locale (accessible sur <http://10.0.4.1:8888>).